

A Resident-Streaming Roofline Model for Compute-in-Memory

Hanyong Shao^{1,2}, Kechao Tang^{†1,3}, and Ru Huang^{†1,3}

¹School of Integrated Circuits & ²Academy for Advanced Interdisciplinary Studies, Peking University, Beijing 100871, China ³Beijing Advanced Innovation Center for Integrated Circuits, Beijing 100871, China [†]Corresponding authors Email: {tkch, ruhuang}@pku.edu.cn

Abstract—From CPUs to GPUs, the Roofline model has become a textbook tool for revealing a platform’s performance ceiling and how well a workload matches it. Compute-in-memory (CIM), however, falls outside this picture because it violates the core Roofline assumption of separate compute and memory units. We present a resident-streaming Roofline model for CIM, in which streaming-activation and resident-weight throughputs replace compute throughput and memory bandwidth, respectively. This follows from a symmetry we observe: conventional processors pair separated hardware with homogeneous data, whereas CIM pairs unified hardware with heterogeneous data—operands residing in the array and operands streaming through it. Using NeuroSim with literature-extracted parameters, we evaluate various CIM technologies, from SRAM to emerging non-volatile memories, on this Roofline and quantify the demands of mainstream large language models across computation stages. With similar periphery, the streaming throughputs of these technologies differ by a factor of only about 10^2 , whereas their resident throughputs, limited by each device’s write mechanism, differ by over 10^4 . The memory thus decides mainly how much reuse CIM needs, from about 2–10 input vectors per weight load on SRAM and eDRAM to about 4×10^4 on NOR flash, making the reuse a workload offers an explicit criterion for choosing among technologies.

Index Terms—Compute-in-memory, roofline model, streaming intensity, data reuse, large language models.

I. INTRODUCTION

The textbook Roofline model [1] relates hardware capability to workload demand through the Arithmetic Intensity (AI), and exposes the bottleneck arising from a mismatch between software and hardware. It describes the capability limits on computation and on data supply by the compute capability π and the memory bandwidth β , respectively. AI represents the number of operations a workload supports per byte of data transferred; comparing it with the hardware ridge point $AI^* = \pi/\beta$ distinguishes the memory-bound region from the compute-bound region and provides theoretical guidance for optimization [2], [3].

The validity of this model rests on an assumption implicit in the von Neumann architecture: memory and compute are separate units [4]. Fig. 1(a) shows the familiar picture. Storage and computation are carried out by different hardware, and although activations A and weights W play different roles in the algorithm, the hardware treats both simply as operands delivered to the compute units: neither has a distinct identity, and the two are interchangeable.

However, this assumption no longer holds in Compute-in-Memory (CIM), a non-von Neumann architecture [5]. In Fig. 1(b), the CIM macro that stores W also calculates A , so the two hardware units are unified [6]. The *weights* take part in computation as *resident state*, while each new *activation* enters through the read path and *streams across* the macro, where it is calculated against that state. CIM turns one class of operands, the weights, into stored state at the very site of

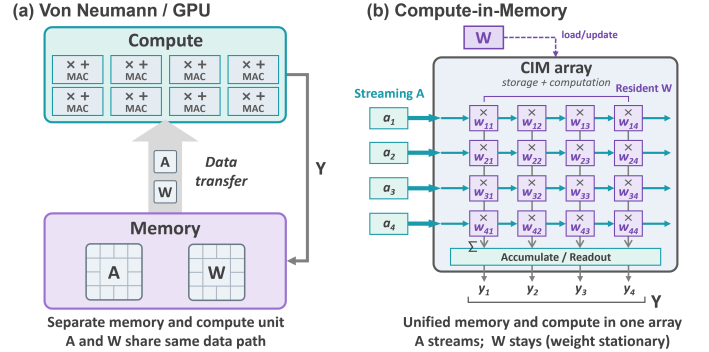


Fig. 1. Hardware schematics: (a) CPU/GPU: separated hardware with homogeneous data; (b) CIM: unified hardware with heterogeneous data.

calculation, while the other class, the activations, flows through the macro as inputs served by that state. The separate hardware units of memory and compute in CPU/GPU disappear, but new roles of streaming and resident operands emerge in CIM.

This change calls for a new Roofline model that rethinks how compute capability and data transfer relate in CIM. The need has two roots. First, the hardware fusion of the CIM architecture dissolves the concept of compute capability π : the capability of a CIM macro to complete multiply-accumulate (MAC) is, in essence, the “transfer rate” of A . Second, the split of operand roles creates two distinct throughputs: weights enter through the write path, which erases, programs, or refreshes the cells, whereas activations enter through the read path, which applies them as inputs and accumulates their products with the weights as outputs. Hence, whereas AI captures the GPU bottleneck as an imbalance between computation and memory access, the CIM bottleneck is the imbalance between these two operand throughputs. Prior studies have recognized this problem [7] and proposed several Roofline analyses for CIM [2], [8]–[14], yet none of them reveals this difference.

Therefore, we propose a resident-streaming Roofline model for CIM. Within an explicit workload window, the model separately counts the input bytes Q_S consumed by streaming calculation and the write bytes Q_R that update or reload resident state, and defines the Streaming Intensity $SI = Q_S/Q_R$ as the streaming demand supported per byte of resident write. Q_S is served by the streaming throughput ρ and Q_R by the resident throughput τ , so the model keeps the form of the classical Roofline, with ρ , τ , and SI taking the places of π , β , and AI. By distinguishing the two operand roles, streaming A and resident W , the model quantifies the imbalance between resident updates and streaming calculation: it bounds the attainable throughput by $P \leq \min(\rho, \tau SI)$, and its ridge $SI^* = \rho/\tau$ marks where the bottleneck shifts from resident updates to streaming calculation.

Building on this model, we carry out a quantitative analysis

from two sides: hardware capability and algorithm demand. On the hardware side, we characterize the streaming and resident throughputs of ten CIM technologies through NeuroSim simulations calibrated with parameters extracted from the literature. On the algorithm side, we examine the matrix multiplications in mainstream large language models (LLM) and characterize the demands posed by static model weights and by the dynamic key-value (KV) cache. The analysis shows that, with similar periphery, the streaming throughputs of the ten technologies differ by a factor of only about 10^2 , whereas their resident throughputs differ by over 10^4 . Hence the ridge SI^* , i.e., the streaming bytes each resident byte must serve before writes stop limiting throughput, is set by the write mechanism of each memory and varies by over 10^4 across technologies. Comparing the SI of a workload with SI^* thus shows which technologies can run it at their streaming ceiling and how far the others fall short, providing clear guidance for matching CIM technologies to workloads.

II. PROPOSED RESIDENT-STREAMING ROOFLINE

A. Workload Demands and Hardware Throughputs

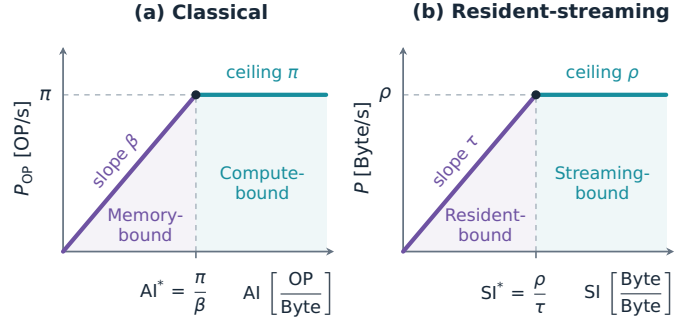
The model builds on the operand roles in Fig. 1(b): a streaming operand enters through the read path and is calculated against resident state, which the write path establishes in the cells. A *workload window* specifies the input work under analysis, the resident state at its start, and the resident writes within it; it defines the work rather than its duration.

Within the window, the model counts two demands. The streaming demand Q_S is the number of streaming input bytes received and served at the chosen logical input boundary. The resident demand Q_R is the number of logical bytes actually written to establish, update, or reload resident state ready for calculation. An initial load counts only if it occurs within the window, but every update or reload inside the window is counted, so Q_R is neither the array capacity nor the unique weight size. The resident operand need not be a weight; in attention, the KV cache of earlier tokens can be resident.

Each demand has a matching hardware throughput. The streaming throughput ρ is not the raw bandwidth of an input bus but the maximum rate at which the hardware serves logical input bytes while completing the specified calculation and outputs. The resident throughput τ is not the rate of applying program pulses alone but the maximum rate at which logical resident bytes become ready for calculation under a given write mode. Each demand and its throughput use the same analysis scope, logical boundary, and precision configuration, although the two roles may use different byte widths. We define

$$SI = \frac{Q_S}{Q_R}, \quad P = \frac{Q_S}{T} \quad [\text{Byte/s}]. \quad (1)$$

Here, T is the window completion time, and P is its average streaming-input throughput; SI is the streaming demand per byte of resident write, taken as infinite for $Q_S > 0$ and $Q_R = 0$. Thus SI and P describe the window, whereas ρ and τ are limits set by the configuration and mode.



Aspect	GPU (a)	CIM (b)
Decomposition	Hardware resources	Data roles
Workload	OP, Byte	Q_S, Q_R
Service rates	π [OP/s], β [Byte/s]	ρ, τ [Byte/s]
Intensity	$AI = OP/Byte$	$SI = Q_S/Q_R$
Vertical axis	$P_{OP} = OP/T$	$P = Q_S/T$
Upper bound	$P_{OP} \leq \min(\pi, \beta AI)$	$P \leq \min(\rho, \tau SI)$
Left regime	Memory-bound	Resident-bound
Right regime	Compute-bound	Streaming-bound
Ridge	$AI^* = \pi/\beta$	$SI^* = \rho/\tau$

Fig. 2. Classical GPU [1] and resident-streaming CIM Rooflines with their accounting correspondence. Panels use independent linear scales; curves are upper bounds. OP and Byte denote operation and transferred-byte counts.

Demands count logical bytes, while every physical step needed to complete the logical service enters the service time of the corresponding path: bit-serial cycles and repeated conversions lengthen the streaming service time, and program and verify steps the resident service time. Output bytes do not add to Q_S , but completing the specified outputs remains part of the service that ρ describes. Maintenance such as refresh is not a workload update; its cost belongs to the corresponding service cycle or available throughput. A repeated input is counted again when it re-enters the chosen logical input boundary. At a macro input, replay across tiles thus adds to Q_S ; at a matrix-stage entry that counts a shared input once, internal replay instead enters the service time and lowers the effective ρ , so it is never charged twice. Likewise, τ depends on the write mode: a complete load and a small KV append need not share one effective value.

B. Throughput Bound and Roofline Interpretation

Each path imposes a necessary service time on the window: the streaming demand needs at least Q_S/ρ , and the resident demand at least Q_R/τ . The window cannot complete before both services are done, so T is at least the larger of the two. With $P = Q_S/T$ and $Q_S/Q_R = SI$, this lower bound on time becomes an upper bound on throughput:

$$T \geq \max\left(\frac{Q_S}{\rho}, \frac{Q_R}{\tau}\right), \quad (2)$$

$$P \leq \min(\rho, \tau SI), \quad SI^* = \frac{\rho}{\tau}. \quad (3)$$

The streaming term caps P at ρ regardless of the resident demand. The resident term rises with SI: each resident byte takes at least $1/\tau$ of window time and is accompanied by SI streaming bytes, so $P \leq \tau SI$.

The two terms meet at the ridge SI^* , the ratio of demands that the hardware requires: the streaming bytes each byte of resident write must support before resident service stops being the tighter constraint. At $SI = SI^*$, the two required service times are equal. For $SI < SI^*$, the resident term is tighter and the window is resident-bound; for $SI > SI^*$, the streaming term is tighter and the window is streaming-bound. Bound (2) does not require the two services to overlap completely; shared resources, execution dependencies, and scheduling determine how closely a window approaches (3).

Fig. 2 places this bound beside the classical Roofline. Both models pair each workload demand with the throughput that serves it and share the slope–ceiling geometry; they differ in the work counted and the physical service behind it, namely operations and memory traffic in the classical model versus streaming input and resident writes here. The vertical axis of Fig. 2(b) thus counts input bytes per second, and SI on its horizontal axis is a byte-to-byte ratio.

Within one execution, P converts directly to useful operations per second, P_{OP} . A logical 128×128 INT8 matrix-vector multiplication takes 128 input bytes and performs 16,384 MACs, or 32,768 useful operations (OP) at two per MAC, so $P_{OP} = (256 \text{ OP/Byte}) P$ for this execution. The factor follows from its shape, precision, and counting convention; it is not a hardware constant shared by all workloads.

C. From Streaming Intensity to Reuse

The ridge can be expressed as the number of input vectors that one resident load must support. On the workload side, consider a resident matrix $W[N, K]$ with output dimension N and input dimension K , and let the reuse count U be the number of input vectors served after one complete load, including the first use. These vectors may accumulate across batches or requests, so U is not the batch size. With b_S and b_R bytes per streaming and resident element, each input vector adds Kb_S to Q_S , whereas the load contributes a fixed NKb_R to Q_R :

$$Q_S = UKb_S, \quad Q_R = NKb_R, \quad SI = \frac{Ub_S}{Nb_R}. \quad (4)$$

At equal byte widths, $SI = U/N$. Table I evaluates this relation for five logical matrices. Because K enlarges both the input vector and the resident matrix, it cancels from the demand ratio, so matrices with the same N share the same SI ; the cancellation concerns the demands, not the hardware that serves them.

TABLE I
STREAMING INTENSITY AFTER ONE COMPLETE WEIGHT LOAD

Weight shape $N \times K$	Reuse count U					
	128	1K	16K	128K	1M	$\rightarrow \infty$
128×128	1.0	8.0	128.0	1024.0	8192.0	∞
1024×1024	0.125	1.0	16.0	128.0	1024.0	∞
4096×4096	0.0313	0.250	4.0	32.0	256.0	∞
1024×4096	0.125	1.0	16.0	128.0	1024.0	∞
4096×1024	0.0313	0.250	4.0	32.0	256.0	∞

One byte/element: $Q_S = UK$, $Q_R = NK$, $SI = U/N$. As $U \rightarrow \infty$, the single load $Q_R = NK$ stays finite. Shapes are logical matrices, not hardware tiles. 1K = 1024, 1M = 1024².

The intensity of a matrix is therefore not fixed; it depends on how the window uses resident state. For the 128×128 INT8 example, loading once and then reusing gives $SI = U/128$, which grows with the reuse each load supports. Reloading before every input limits each load to $U = 1$, so SI stays at $1/128$ however many inputs the window serves. A matrix already resident at the start and not rewritten gives $Q_R = 0$ and $SI = \infty$, so the bound reduces to $P \leq \rho$. Unbounded reuse after one finite load, $U \rightarrow \infty$, approaches the same bound but keeps $Q_R = NKb_R$; it counts a different window. At fixed ρ and τ , the residency policy thus determines SI and the corresponding throughput bound.

On the hardware side, let Δ_S be the steady-state service interval of one complete input vector for the same matrix, and T_R the service time of one complete load that makes its resident state ready for calculation. The two throughputs, and the *critical reuse* obtained by setting $SI = SI^*$, are

$$\rho = \frac{Kb_S}{\Delta_S}, \quad \tau = \frac{NKb_R}{T_R}, \quad (5)$$

$$U^* = \frac{T_R}{\Delta_S} = \frac{Nb_R}{b_S} SI^*. \quad (6)$$

At equal byte widths, $U^* = NSI^*$. In these terms, the two service times in (2) are $Q_S/\rho = U\Delta_S$ and $Q_R/\tau = T_R$, so the two service requirements balance at $U\Delta_S = T_R$: a load-once window is resident-bound for $U < U^*$ and streaming-bound for $U > U^*$. The reuse U thus reflects how the workload uses resident state, and the critical reuse U^* how fast the hardware establishes that state relative to how fast it serves inputs.

For this load-once reference, (3) also bounds the throughput relative to the streaming ceiling:

$$\frac{P}{\rho} \leq \min\left(1, \frac{SI}{SI^*}\right) = \min\left(1, \frac{U}{U^*}\right). \quad (7)$$

Insufficient reuse thus does more than mark a window as resident-bound: it caps the throughput at a fraction U/U^* of the streaming ceiling. Section III supplies both sides of this comparison: Δ_S and T_R for the native configuration of each technology, and the streaming intensities of LLM matrix stages.

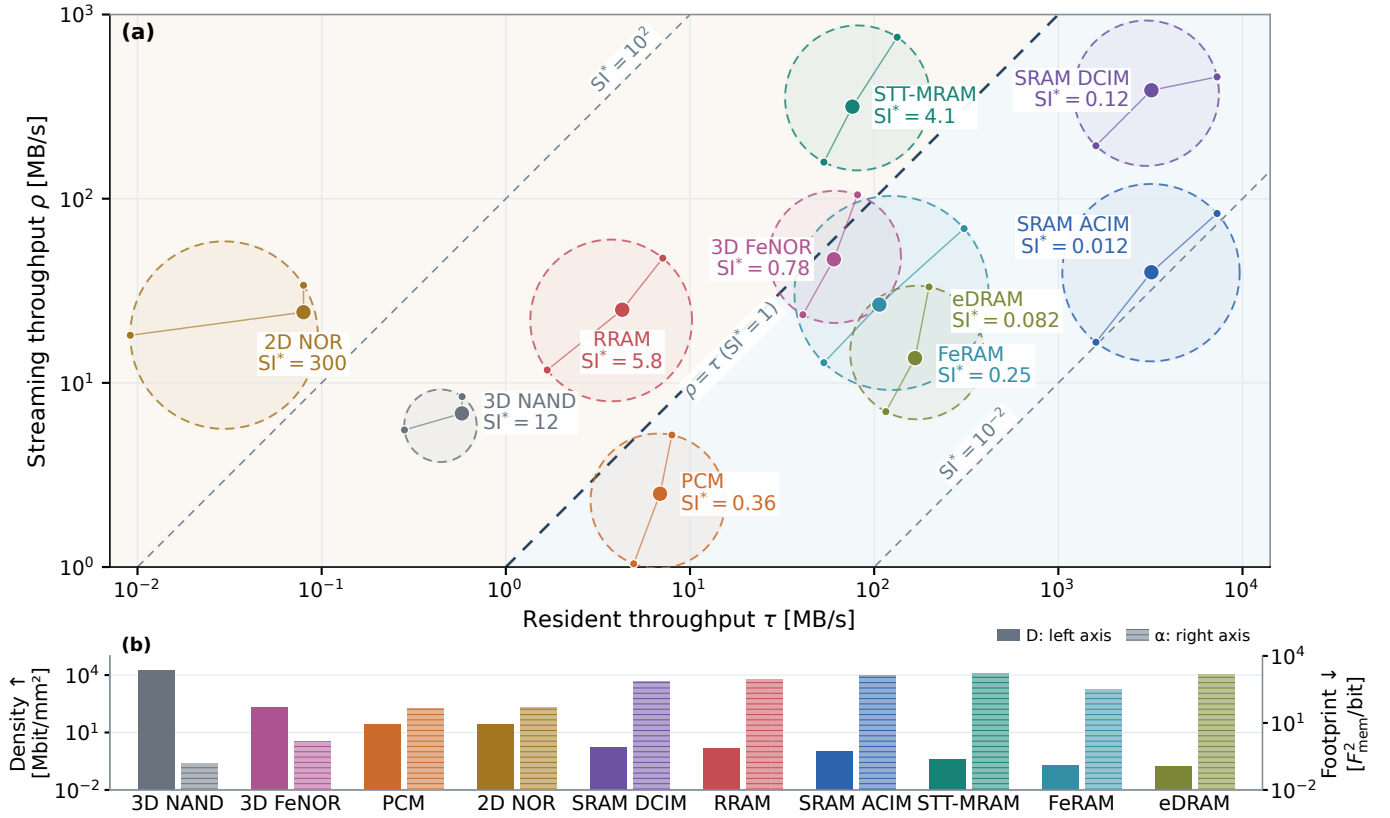


Fig. 3. (a) Streaming ρ and resident τ throughputs for ten native configurations; 1 MB/s = 10^6 Byte/s. Large/small points denote typical/paired alternative scenarios. Circles summarize only these finite scenarios; $\text{SI}^* = 1$ is a ratio reference, not a workload boundary. (b) Binary cell/tile references, ordered by density D (dark bars, left axis), with normalized bit footprint α (finely ruled bars, right axis). Geometry includes reported values and conditional estimates; NAND uses a 500-layer projection. F_{mem} uses nominal nodes except NAND/FeNOR directional half-pitches. Storage references exclude macro overhead and need not match the configurations in (a).

TABLE II
STREAMING INTENSITIES OF REPRESENTATIVE INFERENCE MATRIX STAGES

Model	Weight projections						Attention					
	QKV (U)			FFN/MoE (U)			Prefill (L)			Decode (L)		
	1K	128K	1M	16	1K	16K	1K	8K	64K	1K	8K	64K
Qwen3.5-2B	0.200	25.6	204.8	0.00347	0.222	3.6	6.0	34.0	258.0	10.0	66.0	514.0
Ministral 3 8B	0.167	21.3	170.7	0.00167	0.107	1.7	10.0	66.0	514.0	18.0	130.0	1026.0
Qwen3.6-35B-A3B	0.111	14.2	113.8	0.0130	0.833	13.3	12.0	68.0	516.0	20.0	132.0	1028.0
Hy3 (295B)	0.100	12.8	102.4	0.00477	0.306	4.9	20.0	132.0	1028.0	36.0	260.0	2052.0
Ling-1T (1000B)	0.100	12.8	102.4	0.00326	0.208	3.3	20.0	132.0	1028.0	36.0	260.0	2052.0
MiMo-V2.5-Pro (1020B)	0.0377	4.8	38.6	0.00347	0.222	3.6	35.2	214.4	1648.0	60.8	419.2	3286.4

Entries are SI; all roles use one byte/element. QKV includes any gate, excluding O. FFN/MoE counts one dense FFN or one routed expert. Prefill builds KV from empty; decode appends one token to a visible length L . 1K = 1024, 1M = 1024^2 . Ling-1T at 64K uses the fixed extension.

III. HARDWARE AND WORKLOAD CHARACTERIZATION

A. Memory-Based Reference Configurations

Outline. Introduce the ten native analog/digital reference configurations in Fig. 3, simulated with NeuroSim [15], [16] from device and macro sources—SRAM DCIM [17]–[19], SRAM ACIM [7], [20], [21], 2D NOR [22]–[24], 3D NAND [25]–[27], RRAM [28]–[30], MRAM [31]–[33], PCM [34]–[36], FeRAM [37]–[39], eDRAM [40]–[42], and FeNOR [43]–[45]—and shared peripheral assumptions [46]–[48]. Explain the three paired engineering scenarios and complete service accounting,

retaining native matrix dimensions, resources, and update organizations. Read absolute throughput and throughput ratio separately; these are literature-grounded estimates rather than equal-area chip comparisons. Within each scenario, streaming throughputs differ by about 10^2 and resident throughputs by over 10^4 ; SI^* varies by over 10^4 across technologies but by less than $10\times$ across the scenarios of one technology.

BODY TEXT RESERVED

Native configurations and complete service accounting.

Writing round 1: prose not yet drafted.

B. Inference Matrix Workloads

Outline. Use the six fixed model configurations [49]–[54] as structural examples for Table II, counting common inputs once per matrix stage at one byte per element. Explain QKV intensity U/N_{proj} and FFN intensity $U(D + F)/(3DF)$ using output width N_{proj} (including any gate), hidden width D , and dense-FFN or single-expert width F . For grouped-query attention, use the native head sharing and possibly unequal query/key and value widths: prefill creates each token’s KV once, whereas decode appends one token and treats historical KV as resident.

IV. REUSE REQUIREMENTS AND DESIGN IMPLICATIONS

A. Critical Reuse Across Configurations

Outline. Use Fig. 4 to show that typical full-load thresholds span about 1.6 to 38,867 input vectors, then develop one concrete crossing: at the illustrative reuse $U = 128$, STT-MRAM is resident-bound in the short and typical scenarios (about 181 and 133), but streaming-bound in the long scenario (about 95). Add the low-reuse case: weights reloaded after 16 input vectors leave only SRAM and eDRAM streaming-bound, while NOR flash retains $U/U^* < 0.1\%$ of its streaming ceiling. Connect the native threshold to Qwen3.5-2B and MiMo-V2.5-Pro QKV stages using the existing sequential full-tile service reference, for which both sides accumulate the same tile count. Keep this mapping as a component-service bound, with output reduction and other system costs outside its scope.

For native 128×128 tiles, let $m_K = D/128$, $m_N = N_{\text{proj}}/128$, and $M = m_K m_N$:

$$T_{S,\text{comp}} = MU\Delta_S, \quad T_{R,\text{comp}} = MT_R, \quad (8)$$

$$\rho_{\text{eff}} = \rho/m_N, \quad \tau_{\text{eff}} = \tau, \quad (9)$$

$$SI_{\text{eff}}^* = \frac{SI^*}{m_N} = \frac{U^*}{N_{\text{proj}}}. \quad (10)$$

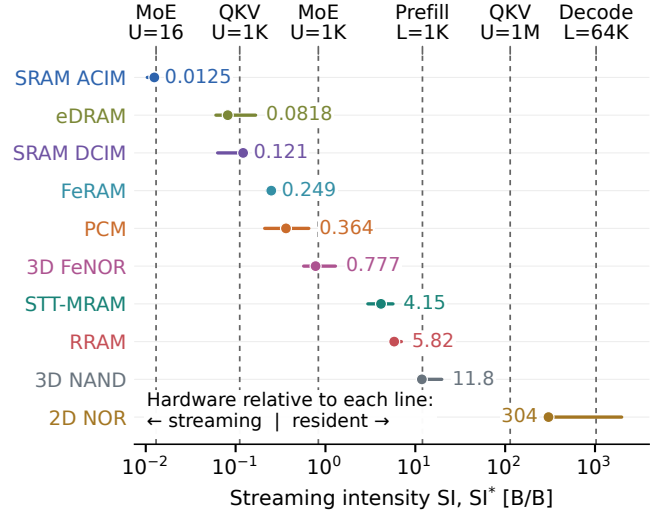
The factor M cancels at service balance. This reference uses one engine, full tiles without padding, and equal U per tile; input replay is included in $T_{S,\text{comp}}$.

BODY TEXT RESERVED

Explain the threshold span and one scenario crossing.

Writing round 1: prose not yet drafted.

Qwen3.6-35B-A3B



● Reference; segment: three native scenarios.

Fig. 4. Full-load critical reuse $U^* = T_R/\Delta_S = NSI^*$ at equal byte widths; each row uses its native $W[N, K]$. Numbers give typical values; segments span only three paired scenarios. Dashed lines mark illustrative reuse counts, not universal balance points. $1K = 1024$.

B. Design Implications

Outline. Turn a target reuse count into an allowable full-load service time using (11), making Fig. 4 a design requirement rather than a ranking. Revisit Fig. 3 to explain why a lower threshold alone does not imply higher performance: streaming throughput still sets the upper plateau. Contrast weight residency with growing attention state in Table II; KV append requires its own update-service boundary before applying any full-load threshold.

$$T_R \leq U_{\text{target}}\Delta_S \iff U_{\text{target}} \geq U^*. \quad (11)$$

This is a necessary service-balance condition for reaching the streaming ceiling, not a guarantee of realized throughput.

BODY TEXT RESERVED

Turn a target reuse count into a resident-service requirement.

Writing round 1: prose not yet drafted.

V. CONCLUSION

Outline. Close with the role-based connection between in-window resident updates and streaming evaluation established by (3). Use Fig. 4 to emphasize the practical outcome: hardware service asymmetry becomes an explicit reuse requirement for residency and update decisions.

BODY TEXT RESERVED

Final conclusion: one short paragraph.

Writing round 1: prose not yet drafted.

REFERENCES

- [1] S. Williams *et al.*, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [2] M. Verhelst *et al.*, “How to Keep Pushing ML Accelerator Performance? Know Your Rooflines!” *IEEE J. Solid-State Circuits*, vol. 60, no. 6, pp. 1888–1905, 2025.
- [3] G. Gerogiannis *et al.*, “DECA: A Near-Core LLM Decompression Accelerator Grounded on a 3D Roofline Model,” in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2025, pp. 184–200.
- [4] S. Rheindt *et al.*, “X-Centric: A Survey on Compute-, Memory- and Application-Centric Computer Architectures,” in *Proc. Int. Symp. Memory Syst. (MEMSYS)*, 2020, pp. 178–193.
- [5] Z. Sun *et al.*, “A Full Spectrum of Computing-in-Memory Technologies,” *Nature Electron.*, vol. 6, no. 11, pp. 823–835, 2023.
- [6] N. Verma *et al.*, “In-Memory Computing: Advances and Prospects,” *IEEE Solid-State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, 2019.
- [7] H. Jia *et al.*, “Scalable and Programmable Neural Network Inference Accelerator Based on In-Memory Computing,” *IEEE J. Solid-State Circuits*, vol. 57, no. 1, pp. 198–211, 2022.
- [8] G. Krishnan *et al.*, “Design Limits of In-Memory Computing: Beyond the Crossbar,” in *Proc. IEEE Electron Devices Technol. Manuf. Conf. (EDTM)*, 2021, pp. 1–3.
- [9] A. Garofalo *et al.*, “A Heterogeneous In-Memory Computing Cluster for Flexible End-to-End Inference of Real-World Deep Neural Networks,” *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 12, no. 2, pp. 422–435, 2022.
- [10] J. Sun *et al.*, “Analog or Digital In-Memory Computing? Benchmarking Through Quantitative Modeling,” in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Des. (ICCAD)*, 2023, pp. 1–9.
- [11] F.-H. Meng *et al.*, “Compute-In-Memory Technologies for Deep Learning Acceleration,” *IEEE Nanotechnol. Mag.*, vol. 18, no. 1, pp. 44–52, 2024.
- [12] T. Sharma *et al.*, “What, When, Where to Compute-in-Memory for Efficient Matrix Multiplication During Machine Learning Inference,” *IEEE Trans. Emerg. Topics Comput.*, vol. 13, no. 3, pp. 1215–1229, 2025.
- [13] K. Zhou *et al.*, “A Full-System, Programmable, and Extensible In-Memory Computing Simulation Framework for Deep Learning,” in *Proc. ACM/IEEE Design Autom. Conf. (DAC)*, 2025, pp. 1–7.
- [14] H. Liu *et al.*, “Harnessing Memory Technology Innovations for On-Device Large Language Models in Augmented Reality SoCs: Cooptimizing System Architecture with Model Characteristics,” *IEEE Solid-State Circuits Mag.*, vol. 17, no. 4, pp. 39–46, 2025.
- [15] J. Lee *et al.*, “NeuroSim V1.4: Extending Technology Support for Digital Compute-in-Memory Toward 1nm Node,” *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 4, pp. 1733–1744, 2024.
- [16] X. Peng *et al.*, “DNN+NeuroSim: An End-to-End Benchmarking Framework for Compute-in-Memory Accelerators with Versatile Device Technologies,” in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2019, pp. 32.5.1–32.5.4.
- [17] J. Oh *et al.*, “D6CIM: 60.4-TOPS/W, 1.46-TOPS/mm², 1005-Kb/mm² Digital 6T-SRAM-Based Compute-in-Memory Macro Supporting 1-to-8b Fixed-Point Arithmetic in 28-nm CMOS,” in *Proc. ESSCIRC*, 2023, pp. 413–416.
- [18] B. Yan *et al.*, “A 1.041-Mb/mm² 27.38-TOPS/W Signed-INT8 Dynamic-Logic-Based ADC-less SRAM Compute-in-Memory Macro in 28nm with Reconfigurable Bitwise Operation for AI and Embedded Applications,” in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2022, pp. 188–190.
- [19] Y. Yuan *et al.*, “A 28-nm Digital Transpose SRAM Compute-in-Memory Macro With Accurate/Approximate Dual Mode for Floating-Point Edge Training and Inference,” *IEEE J. Solid-State Circuits*, vol. 61, no. 9, pp. 5098–5110, 2026.
- [20] K. Xiao *et al.*, “A 28nm 32Kb SRAM Computing-in-Memory Macro With Hierarchical Capacity Attenuator and Input Sparsity-Optimized ADC for 4b Mac Operation,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 6, pp. 1816–1820, 2023.
- [21] J.-W. Su *et al.*, “8-Bit Precision 6T SRAM Compute-in-Memory Macro Using Global Bitline-Combining Scheme for Edge AI Chips,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 4, pp. 2304–2308, 2024.
- [22] Infineon Technologies, *S29GL01GS, S29GL512S, S29GL256S, S29GL128S — 128 Mb / 256 Mb / 512 Mb / 1 Gb GL-S MIRRORBIT Flash, Parallel, 3.0 V, Military*, 2024, datasheet 002-18741 Rev. *E.
- [23] Winbond Electronics, *W25Q128JV — 3V 128M-Bit Serial Flash Memory with Dual/Quad SPI*, 2019, datasheet Rev. G.
- [24] X. Guo *et al.*, “Fast, Energy-Efficient, Robust, and Reproducible Mixed-Signal Neuromorphic Classifier Based on Embedded NOR Flash Memory Technology,” in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2017, pp. 6.5.1–6.5.4.
- [25] W. Shim *et al.*, “System-Technology Codesign of 3-D NAND Flash-Based Compute-in-Memory Inference Engine,” *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 7, no. 1, pp. 61–69, 2021.
- [26] H.-T. Lue *et al.*, “Optimal Design Methods to Transform 3D NAND Flash into a High-Density, High-Bandwidth and Low-Power Nonvolatile Computing in Memory (nvCIM) Accelerator for Deep-Learning Neural Networks (DNN),” in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2019, pp. 38.1.1–38.1.4.
- [27] Macronix International Co., Ltd., *MX30LF1G18AC: 3V, 1G-bit NAND Flash Memory*, 2016, datasheet PM2133 Rev. 1.2.
- [28] W. Ye *et al.*, “A 28-nm RRAM Computing-in-Memory Macro Using Weighted Hybrid 2T1R Cell Array and Reference Subtracting Sense Amplifier for AI Edge Inference,” *IEEE J. Solid-State Circuits*, vol. 58, no. 10, pp. 2839–2850, 2023.
- [29] W. Wan *et al.*, “A compute-in-memory chip based on resistive random-access memory,” *Nature*, vol. 608, no. 7923, pp. 504–512, 2022.
- [30] Y.-C. Chiu *et al.*, “A 40nm 2Mb ReRAM Macro with 85% Reduction in FORMING Time and 99% Reduction in Page-Write Time Using Auto-FORMING and Auto-Write Schemes,” in *Proc. Symp. VLSI Technol.*, 2019, pp. T232–T233.
- [31] H. Li *et al.*, “A lossless and fully parallel spintronic compute-in-memory macro for artificial intelligence chips,” *Nature Electron.*, vol. 8, no. 11, pp. 1046–1058, 2025.
- [32] Q. Dong *et al.*, “A 1-Mb 28-nm 1T1MTJ STT-MRAM With Single-Cap Offset-Cancelled Sense Amplifier and In Situ Self-Write-Termination,” *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 231–239, 2019.
- [33] T.-H. Yang *et al.*, “A 28nm 32Kb Embedded 2T2MTJ STT-MRAM Macro with 1.3ns Read-Access Time for Fast and Reliable Read Applications,” in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2018, pp. 482–484.
- [34] W.-S. Khwa *et al.*, “A 40-nm, 2M-Cell, 8b-Precision, Hybrid SLC-MLC PCM Computing-in-Memory Macro with 20.5–65.0TOPS/W for Tiny-AI Edge Devices,” in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2022, pp. 1–3.
- [35] M. Le Gallo *et al.*, “A 64-core mixed-signal in-memory compute chip based on phase-change memory for deep neural network inference,” *Nature Electron.*, vol. 6, no. 9, pp. 680–693, 2023.
- [36] P. Narayanan *et al.*, “Fully On-Chip MAC at 14 nm Enabled by Accurate Row-Wise Programming of PCM-Based Weights and Parallel Vector-Transport in Duration-Format,” *IEEE Trans. Electron Devices*, vol. 68, no. 12, pp. 6629–6636, 2021.
- [37] J. Okuno *et al.*, “SoC Compatible 1T1C FeRAM Memory Array Based on Ferroelectric Hf_{0.5}Zr_{0.5}O₂,” in *Proc. Symp. VLSI Technol.*, 2020, pp. 1–2.
- [38] N. Ramaswamy *et al.*, “NVDram: A 32Gb Dual Layer 3D Stacked Non-volatile Ferroelectric Memory with Near-DRAM Performance for Demanding AI Workloads,” in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2023, pp. 1–4.
- [39] M. Sung *et al.*, “Low Voltage and High Speed 1Xnm 1T1C FE-RAM with Ultra-Thin 5nm HZO,” in *Proc. IEEE Int. Electron Devices Meeting (IEDM)*, 2021, pp. 33.3.1–33.3.4.
- [40] J. Song *et al.*, “A 4-bit Calibration-Free Computing-In-Memory Macro With 3T1C Current-Programmed Dynamic-Cascade Multi-Level-Cell eDRAM,” *IEEE J. Solid-State Circuits*, vol. 59, no. 3, pp. 842–854, 2024.
- [41] R. Gitterman *et al.*, “An 800-MHz Mixed-V_T 4T IFGC Embedded DRAM in 28-nm CMOS Bulk Process for Approximate Storage Applications,” *IEEE J. Solid-State Circuits*, vol. 53, no. 7, pp. 2136–2148, 2018.
- [42] P.-C. Wu *et al.*, “An Integer-Floating-Point Dual-Mode Gain-Cell Computing-in-Memory Macro for Advanced AI Edge Chips,” *IEEE J. Solid-State Circuits*, vol. 60, no. 1, pp. 158–170, 2025.
- [43] Y. Zhou *et al.*, “3D Vertical FeFET Array with Record Endurance (> 10¹²), Fast Writing (±2V, 20 ns), Disturb Immunity, and Kb-scale Verification for High Density 1T RAM,” in *Proc. Symp. VLSI Technol. Circuits*, 2026, pp. 1–3, paper T11.1.
- [44] Y. Zhou *et al.*, “3D NOR-Type FeFETs with Record Endurance of 10¹¹, Fast Erase of 50 ns, and Immediate Read-After-Write for In-Memory Learning,” in *Proc. Symp. VLSI Technol. Circuits*, 2025, pp. 1–3.
- [45] Y. Feng *et al.*, “Efficient Large Scale Neural Network Acceleration With 3-D FeNOR-Based Computing-in-Memory Design,” *IEEE Trans. Electron Devices*, vol. 72, no. 5, pp. 2319–2326, 2025.
- [46] F. Tang *et al.*, “A 28 nm CMOS 10 bit 100 MS/s Asynchronous SAR ADC with Low-Power Switching Procedure and Timing-Protection Scheme,” *Electronics*, vol. 10, no. 22, p. 2856, 2021.
- [47] B. Zimmer *et al.*, “SRAM Assist Techniques for Operation in a Wide Voltage Range in 28-nm CMOS,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 59, no. 12, pp. 853–857, 2012.
- [48] Y. Yokoyama *et al.*, “Disturbance Aware Dynamic Power Reduction in Synchronous 2RW Dual-Port 8T SRAM by Self-Adjusting Wordline Pulse Timing,” *IEEE J. Solid-State Circuits*, vol. 58, no. 7, pp. 2098–2108, 2023.
- [49] Qwen Team, “Qwen3.5: Towards Native Multimodal Agents,” Qwen blog, Feb. 2026.
- [50] A. H. Liu *et al.*, “Minstral 3,” arXiv:2601.08584, Jan. 2026.
- [51] Qwen Team, “Qwen3.6-35B-A3B: Agentic Coding Power, Now Open to All,” Qwen blog, Apr. 2026.
- [52] Tencent Hy Team, “Introducing Hy3,” Tencent Hy research blog, Jul. 2026.
- [53] Ling Team *et al.*, “Every Activation Boosted: Scaling General Reasoner to 1 Trillion Open Language Foundation,” arXiv:2510.22115, Oct. 2025.
- [54] Xiaomi MiMo Team, “MiMo-V2.5-Pro,” Model card, Hugging Face, 2026.